OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 18

1.Skill18.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_JOBS 100
#define MAX_CMD 256

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
```

```
    int state;
} Job;

/*
  States:
  0 = Running
  1 = Stopped
  2 = Completed
*/

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

/* Add a job */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS) {
```

```
        printf("Job table full!\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;

    strncpy(jobs[job_count].command, command, MAX_CMD -
1);
    jobs[job_count].command[MAX_CMD - 1] = '\0';
    jobs[job_count].state = 0;

    printf("[%d] PID=%d Running %s\n",
           jobs[job_count].job_id,
           (int)jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}
```

```
/* Update job states */
void update_jobs(void)
{
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state == 2)
            continue;

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result <= 0)
            continue;
```

```
if (WIFSTOPPED(status)) {
    jobs[i].state = 1;

    printf("[%d] Stopped\t%s\n",
        jobs[i].job_id,
        jobs[i].command);
}
else if (WIFCONTINUED(status)) {
    jobs[i].state = 0;

    printf("[%d] Continued\t%s\n",
        jobs[i].job_id,
        jobs[i].command);
}
else if (WIFEXITED(status) || WIFSIGNALED(status)) {
    jobs[i].state = 2;
```

```
            printf("[%d] Completed\t%s\n",
                jobs[i].job_id,
                jobs[i].command);
        }
    }
}

/* Display jobs */
void show_jobs(void)
{
    update_jobs();

    printf("\n------ JOB TABLE ------\n");

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state == 0) {
            printf("[%d] PID=%d Running %s\n",
                jobs[i].job_id,
```

```
            (int)jobs[i].pid,
            jobs[i].command);
    }
    else if (jobs[i].state == 1) {
        printf("[%d] PID=%d Stopped %s\n",
            jobs[i].job_id,
            (int)jobs[i].pid,
            jobs[i].command);
    }
    else {
        printf("[%d] PID=%d Completed %s\n",
            jobs[i].job_id,
            (int)jobs[i].pid,
            jobs[i].command);
    }
}

printf("---------------------\n");
```

```
}

/* Stop a job */
void stop_job(int job_id)
{
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].job_id == job_id && jobs[i].state == 0) {
            if (kill(jobs[i].pid, SIGSTOP) == 0) {
                jobs[i].state = 1;
                printf("[%d] Stopped\n", job_id);
            }
            else {
                perror("SIGSTOP");
            }

            return;
        }
    }
```

```
    printf("Running job [%d] not found.\n", job_id);
}

/* Continue stopped job */
void continue_job(int job_id)
{
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].job_id == job_id && jobs[i].state == 1) {
            if (kill(jobs[i].pid, SIGCONT) == 0) {
                jobs[i].state = 0;
                printf("[%d] Continued\n", job_id);
            }
            else {
                perror("SIGCONT");
            }

            return;
```

```
        }
    }

    printf("Stopped job [%d] not found.\n", job_id);
}

int main(void)
{
    char input[MAX_CMD];

    printf("================================\n");
    printf(" OSSP Program 18 - Signal Jobs\n");
    printf("================================\n");

    printf("Commands:\n");
    printf(" jobs       -> list jobs\n");
    printf(" stop ID    -> stop a job\n");
    printf(" cont ID    -> resume a stopped job\n");
```

```
printf(" exit       -> exit\n\n");

while (1) {
    update_jobs();

    printf("shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    /* Exit */
    if (strcmp(input, "exit") == 0)
```

```
    break;

/* Jobs */
if (strcmp(input, "jobs") == 0) {
    show_jobs();
    continue;
}

/* Stop job */
if (strncmp(input, "stop ", 5) == 0) {
    int id = atoi(input + 5);
    stop_job(id);
    continue;
}

/* Continue job */
if (strncmp(input, "cont ", 5) == 0) {
    int id = atoi(input + 5);
```

```
    continue_job(id);
    continue;
}

/* Check background & */
int background = 0;
int len = (int)strlen(input);

if (len > 0 && input[len - 1] == '&') {
    background = 1;
    input[--len] = '\0';

    while (len > 0 &&
           (input[len - 1] == ' ' ||
            input[len - 1] == '\t')) {
        input[--len] = '\0';
    }
}
```

```
if (input[0] == '\0')
    continue;

/* Save original command */
char command_copy[MAX_CMD];

strncpy(command_copy, input, MAX_CMD - 1);
command_copy[MAX_CMD - 1] = '\0';

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

/* Child process */
```

```
if (pid == 0) {
    setpgid(0, 0);

    char *args[64];
    int argc = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && argc < 63) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }

    args[argc] = NULL;

    if (argc == 0)
        _exit(0);
```

```
    execvp(args[0], args);


    perror("execvp");
    _exit(1);
}
/* Parent process */
else {
    if (background) {
        setpgid(pid, pid);
        add_job(pid, command_copy);
    }
    else {
        int status;


        if (waitpid(pid, &status, 0) == -1)
            perror("waitpid");
    }
}
```

```
    }

    printf("\nExiting...\n");

    /* Clean up remaining jobs */
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state != 2) {
            kill(jobs[i].pid, SIGTERM);
            waitpid(jobs[i].pid, NULL, 0);
        }
    }

    return 0;
}
```

OUTPUT :

```
sadwika@DESKTOP-8CQAO1L:~$ nano skill18.c
sadwika@DESKTOP-8CQAO1L:~$ gcc -Wall -Wextra -g skill18.c -o skill18
sadwika@DESKTOP-8CQAO1L:~$ ./skill18
================================
 OSSP Program 18 - Signal Jobs
================================
Commands:
 jobs       -> list jobs
 stop ID    -> stop a job
 cont ID    -> resume a stopped job
 exit       -> exit

shell>
```